

"Supervised Learning Approach for Next-Day Temperature and Precipitation Forecasting in Basel"



A project report submitted as part of the coursework for Programming Fundamentals during the 1st Semester, 2025.

Submitted By

Muhammad Zain 2025-BSCPE-118

Supervised By

Dr. Bilal Ahmad

Fall 2025

Computer Engineering

University of Engineering and Technology Lahore, Faisalabad Campus

❖ **Table of Contents**

- **Objectives**
- **Problem Statement**
- **Proposed Solution (ML Trained Model)**
- **Dataset Information**
- **Libraries Information**
- **Visualizations / Graphs**
 - 6.1 Histogram: Basel Temperature
 - 6.2 Histogram: Basel Precipitation
 - 6.3 Histogram: Basel Humidity 6.4 Boxplot: Temperature by Month
 - 6.5 Boxplot: Precipitation by Month
 - 6.6 Countplot: Days per Month
 - 6.7 Time Series: Temperature Forecast (Last 200 Days)
 - 6.8 Time Series: Precipitation Forecast (Last 200 Days)
 - 6.9 Scatter Plot: Actual vs Predicted Temperature
 - 6.10 Scatter Plot: Actual vs Predicted Precipitation
 - 6.11 Horizontal Bar Plot: Top 15 Feature Importance (Temperature Model)
 - 6.12 Horizontal Bar Plot: Top 15 Feature Importance (Precipitation Model)
- **Step-by-Step Implementation**
 - 7.1 Block 01: Import Libraries
 - 7.2 Block 02: Reading the Data
 - 7.3 Block 03: Feature Engineering
 - 7.4 Block 04: Train-Test Split
 - 7.5 Block 05: Model Training
 - 7.6 Block 06: Generating Predictions
 - 7.7 Block 07: Model Performance Metrics
 - 7.8 Block 08: Visualizations
 - 7.9 Block 09: Save Models
- **Conclusion**

Objectives:

The primary goal of this project is to develop a practical, data-driven machine learning system capable of forecasting next-day weather conditions specifically for Basel, Switzerland, using historical observations from multiple European cities. The objectives are multi-faceted and designed to build a comprehensive understanding of both the data and the machine learning pipeline:

- To thoroughly explore and analyze a large-scale historical weather dataset that includes daily measurements from various European cities, enabling the identification of patterns, trends, seasonal variations, and potential correlations across geographical locations.
- To engineer meaningful features from time-series data, such as lag variables and shifted targets, to transform the sequential nature of weather data into a supervised learning problem suitable for regression models.
- To implement and train Random Forest Regressor models—one dedicated to temperature prediction and another to precipitation—to achieve accurate continuous-value forecasts for the next day in Basel.
- To rigorously evaluate the trained models using standard regression performance metrics, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score, in order to quantify predictive accuracy and identify strengths or limitations.
- To extract and interpret feature importance rankings from the models, providing insights into which weather variables (from Basel and other cities) most strongly influence next-day temperature and precipitation.
- To create a wide range of informative visualizations—covering data distributions, seasonal effects, outlier detection, temporal forecasts, prediction accuracy, and feature contributions—to facilitate intuitive understanding of the data and model behavior for both technical and non-technical audiences.
- To demonstrate end-to-end machine learning workflow best practices, including data preprocessing, model training, evaluation, visualization, and persistence, while ensuring reproducibility through fixed random seeds and modular code structure.

These objectives collectively aim to bridge theoretical concepts from supervised learning with real-world application in environmental forecasting

Problem Statement:

Weather forecasting plays a critical role in numerous sectors, including agriculture (for crop planning and irrigation), transportation (aviation and road safety), energy management (demand forecasting for heating/cooling), disaster preparedness, tourism, and everyday public decision-making. Inaccurate or delayed forecasts can lead to significant economic losses, safety risks, and inefficiencies.

Traditional numerical weather prediction (NWP) models, while powerful, rely on complex physics-based simulations that require supercomputing resources, extensive domain expertise, and large amounts of real-time atmospheric data. These approaches can be

computationally expensive, difficult to customize for specific locations, and sometimes less accessible for smaller-scale or educational applications.

In the context of Basel—a city with a temperate climate influenced by continental and oceanic weather patterns—there exists a need for a lightweight, interpretable, and automated system that can reliably predict key next-day metrics (mean temperature and precipitation) using only historical observational data. Such a system should leverage patterns from interconnected European weather stations, account for temporal dependencies (e.g., how today's conditions affect tomorrow), handle the continuous nature of the targets, and provide transparent insights into influencing factors. This project addresses the gap by proposing a machine learning-based alternative that is easier to implement, interpret, and deploy on standard hardware while delivering competitive predictive performance.

Proposed Solution (ML Trained Model):

The proposed solution employs a supervised machine learning approach using regression to predict two continuous target variables: next-day mean temperature and next-day precipitation in Basel. To handle the multi-output nature of the problem efficiently, two separate Random Forest Regressor models are trained—one for each target. Random Forest was selected due to its robustness, ability to handle high-dimensional data, resistance to overfitting through ensemble averaging, and built-in feature importance scoring.

Key elements of the solution include:

- **Feature Engineering:** Creating lag features (previous day's values) and shifting current values to form tomorrow's targets, capturing temporal autocorrelation essential for time-series forecasting.
- **Data Splitting:** A random train-test split (75-25%) with a fixed random state to ensure reproducibility, allowing the model to learn general patterns without chronological leakage in this educational context.
- **Model Architecture:** Ensemble of 200 decision trees per Random Forest, providing strong generalization and non-linear modeling capabilities without requiring extensive hyperparameter tuning.
- **Evaluation:** Comprehensive metrics (MAE, RMSE, R^2) to assess absolute errors, overall deviation, and explained variance.
- **Interpretability:** Feature importance extraction to reveal which variables (e.g., local humidity, pressure from nearby cities) drive predictions.
- **Visualization and Persistence:** Extensive plots for insight and model saving via Joblib for future inference.

This approach offers an accessible yet powerful alternative to traditional forecasting methods, demonstrating how ensemble techniques can achieve high accuracy (particularly for temperature) while remaining interpretable.

Dataset Information:

The dataset comprises daily weather observations collected from multiple European cities over a 10-year period (2000–2010), providing a rich source of temporal and spatial weather

patterns. It contains approximately 3,654 rows (one per day) and 165 columns, covering a wide range of meteorological variables.

Key features include temperature (mean, min, max), humidity, pressure, precipitation, wind speed, and cloud cover for cities such as Basel, Heathrow, Stockholm, Tours, and others. The multi-city aspect is particularly valuable, as weather systems often span regions, allowing the model to learn cross-location influences (e.g., pressure changes in nearby cities signaling approaching fronts).

Additional derived feature:

- **MONTH:** Extracted from the date for seasonal analysis and visualization.

Target variables:

- **BASEL_temp_mean_tomorrow:** Shifted version of Basel's mean temperature, representing the next day's value to predict.
- **BASEL_precipitation_tomorrow:** Similarly shifted precipitation amount, capturing rainfall/snowfall in mm.

The dataset has no missing values in core columns, enabling straightforward preprocessing. Its time-series structure necessitates careful handling of shifts to avoid data leakage while preserving sequential integrity.

Libraries Information:

The project utilizes several core Python libraries to build a complete machine learning pipeline:

- **Pandas:** For efficient data loading (CSV), manipulation, cleaning, feature engineering (e.g., shifting columns), and exploratory analysis through methods like `head()`, `info()`, and `describe()`.
- **NumPy:** Provides support for numerical computations, array operations, and mathematical functions used in metric calculations.
- **Matplotlib & Seaborn:** Matplotlib serves as the foundational plotting library, while Seaborn offers higher-level statistical visualizations (histograms with KDE, boxplots, countplots) with enhanced aesthetics and built-in themes for professional-looking graphs.
- **Scikit-learn:** The primary machine learning toolkit, providing `RandomForestRegressor` for modeling, `train_test_split` for data partitioning, metrics (mean_absolute_error, mean_squared_error, r2_score) for evaluation, and utilities for feature importance extraction.
- **Joblib:** Enables efficient serialization and saving of trained models to disk, allowing reuse without retraining.

These libraries collectively support an end-to-end workflow from data ingestion to model deployment, emphasizing readability and reproducibility.

Visualizations / Graphs:

A comprehensive set of visualizations was created to explore the data, reveal patterns, detect anomalies, and evaluate model performance:

1. **Histogram of Basel Temperature distribution:** Shows the frequency distribution and density (KDE) of daily mean temperatures, highlighting the multimodal nature due to seasonal changes.
2. **Histogram of Basel Precipitation distribution:** Illustrates the highly skewed distribution of precipitation (many dry days, few heavy rain events).
3. **Histogram of Basel Humidity:** Displays humidity levels, useful for understanding comfort and precipitation correlations.
4. **Boxplot: Temperature by Month:** Reveals seasonal cycles, medians, quartiles, and outliers in temperature across months.
5. **Boxplot: Precipitation by Month:** Highlights monthly rainfall variability and potential wet/dry seasons.
6. **Countplot: Number of Days per Month:** Confirms balanced sampling across months (approximately 30-31 days each).
7. **Time Series: Actual vs Predicted Temperature (last 200 days):** Line plot overlay comparing true and forecasted temperature trends over time, demonstrating tracking ability.
8. **Time Series: Actual vs Predicted Precipitation (last 200 days):** Similar overlay for precipitation, showing challenges in predicting intermittent events.
9. **Scatter Plot: Actual vs Predicted Temperature:** Points clustered around the diagonal line indicate high accuracy.
10. **Scatter Plot: Actual vs Predicted Precipitation:** More scattered due to the variable's sparsity, reflecting lower but realistic performance.
11. **Horizontal Bar Plot: Top 15 Feature Importance (Temperature model):** Ranks variables by influence, often showing local lags and pressure as dominant.
12. **Horizontal Bar Plot: Top 15 Feature Importance (Precipitation model):** Highlights different drivers, such as humidity and regional precipitation.

These visualizations provide deep insights into data characteristics and model behavior.

Step-by-Step Implementation:

(The code blocks and outputs remain as originally provided, but explanations in each block have been expanded for clarity and depth where applicable. The core code structure is unchanged.)

Block 01: Import Libraries Explanation: All essential libraries are imported at the outset to ensure availability throughout the script. Styling configurations and a fixed random seed are applied to guarantee consistent, reproducible visualizations and model behavior across runs.

Block 02: Reading the Data Explanation: The CSV file is loaded into a Pandas DataFrame for efficient handling. Basic exploratory commands provide an overview of structure, summary statistics, and date range. Converting the DATE column to datetime format enables time-based operations and sorting.

Block 03: Feature Engineering Explanation: Temporal features are created using Pandas' shift function: tomorrow's values become targets (supervised learning setup), while yesterday's values (lags) add historical context. Dropping NaN rows ensures clean, complete data for training. Features and targets are separated, with DATE excluded to focus on numerical predictors.

Block 04: Train-Test Split Explanation: A stratified random split preserves the overall data distribution while creating independent training and testing sets. Using the same features for both targets maintains consistency, and a fixed random state ensures identical splits for fair comparison.

Block 05: Model Training Explanation: Two independent Random Forest models are instantiated with 200 trees each—a balance between accuracy and computational efficiency. Fitting on training data allows the ensemble to learn complex non-linear relationships and interactions automatically.

Block 06: Generating Predictions Explanation: Trained models generate forecasts on the unseen test set, simulating real-world deployment where future data features are available but targets are unknown.

Block 07: Model Performance Metrics Explanation: A reusable evaluation function computes and displays MAE (average absolute error in original units), RMSE (penalizing larger errors more heavily), and R^2 (proportion of variance explained). Temperature shows excellent results due to stronger autocorrelation, while precipitation's low R^2 reflects the inherent difficulty of predicting rare events.

Block 08: Visualizations Explanation: Extensive plotting reveals data characteristics (distributions, seasonality), model tracking over time, point-wise accuracy, and key drivers, making abstract performance concrete and interpretable.

Block 09: Save Models Explanation: Persisting models to disk enables future predictions without retraining, completing a production-ready pipeline.

Block 01: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import train_test_split
import joblib

plt.style.use('seaborn-v0_8')
sns.set_palette("husl")
np.random.seed(42)
```

Explanation:

At the beginning of the script, all necessary Python libraries are imported to support the entire machine learning pipeline. Pandas is used for data manipulation and analysis, NumPy for numerical operations, Matplotlib and Seaborn for creating high-quality visualizations with consistent styling, and Scikit-learn for building, training, and evaluating the Random Forest models as well as computing performance metrics. Joblib is included for model persistence. Setting a consistent plotting style (seaborn-v0_8) and color palette ensures that all graphs have a professional and uniform appearance. Additionally, fixing the NumPy random seed to 42 guarantees reproducibility—meaning that random processes (such as train-test splitting and Random Forest tree construction) will produce identical results every time the code is run, which is crucial for debugging, comparison, and academic reporting.

Block 02: Reading the Data

```
df = pd.read_csv("weather_prediction_dataset.csv")
print(df.head())
print(df.tail())
print(df.info())
print(df.describe(include="all"))

df['DATE'] = pd.to_datetime(df['DATE'], format='%Y%m%d')

print("Dataset shape:", df.shape)
print("Date range:", df['DATE'].min().date(), "to",
df['DATE'].max().date())
```

Output:

```
   DATE  MONTH  ...  TOURS_temp_min  TOURS_temp_max
0  20000101      1  ...              7.2             9.8
1  20000102      1  ...              6.6             9.2
2  20000103      1  ...              6.6             9.6
3  20000104      1  ...              6.4            10.8
4  20000105      1  ...              6.4             9.5

[5 rows x 165 columns]
   DATE  MONTH  ...  TOURS_temp_min  TOURS_temp_max
3649  20091228     12  ...           1.8            10.6
3650  20091229     12  ...           6.2            14.5
3651  20091230     12  ...           8.7            11.3
3652  20091231     12  ...           6.2            10.9
3653  20100101      1  ...          -0.7             1.8

[5 rows x 165 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3654 entries, 0 to 3653
```



```
Columns: 165 entries, DATE to TOURS_temp_max
dtypes: float64(150), int64(15)
memory usage: 4.6 MB
None
```

	DATE	MONTH	...	TOURS_temp_min
TOURS_temp_max				
count	3.654000e+03	3654.000000	...	3654.000000
mean	2.004568e+07	6.520799	...	7.860536
std	2.874287e+04	3.450083	...	5.692256
min	2.000010e+07	1.000000	...	-13.000000
25%	2.002070e+07	4.000000	...	3.700000
50%	2.004567e+07	7.000000	...	8.300000
75%	2.007070e+07	10.000000	...	12.300000
max	2.010010e+07	12.000000	...	22.600000

```
[8 rows x 165 columns]
Dataset shape: (3654, 165)
Date range: 2000-01-01 to 2010-01-01
```

Explanation:

Explanation: The dataset is loaded from the CSV file "weather_prediction_dataset.csv" into a Pandas DataFrame, which is the foundational structure for all subsequent data handling. Printing the first few rows (`head()`) and last few rows (`tail()`) provides an immediate visual check of the data structure and confirms that the file was read correctly. The `info()` method displays column names, data types, and memory usage, revealing that the dataset has 165 columns (mostly numerical float64 and some int64) and no obvious missing values in the summary. The `describe()` function generates descriptive statistics (count, mean, standard deviation, min/max, quartiles) for all columns, helping identify the scale, central tendency, and variability of weather variables across cities. Converting the 'DATE' column to a proper datetime format using `pd.to_datetime` enables time-based operations such as sorting, shifting, and plotting over chronological order. Finally, printing the dataset shape and date range confirms the temporal scope (approximately 10 years of daily data from January 1, 2000, to January 1, 2010), ensuring the data covers multiple seasons and years for robust pattern learning.

Block 03: Feature Engineering

```
df['BASEL_temp_mean_tomorrow'] = df['BASEL_temp_mean'].shift(-1)
df['BASEL_precipitation_tomorrow'] = df['BASEL_precipitation'].shift(-1)
```

```
df['BASEL_temp_mean_lag1'] = df['BASEL_temp_mean'].shift(1)
df['BASEL_precipitation_lag1'] = df['BASEL_precipitation'].shift(1)

df = df.dropna().reset_index(drop=True)

X = df.drop(['DATE', 'BASEL_temp_mean_tomorrow',
            'BASEL_precipitation_tomorrow'], axis=1)
y_temp = df['BASEL_temp_mean_tomorrow']
y_precip = df['BASEL_precipitation_tomorrow']
```

Explanation:

This critical step transforms the raw time-series data into a supervised learning format suitable for regression. The target variables are created by shifting the current day's Basel temperature and precipitation values forward by one day using `.shift(-1)`, effectively turning "tomorrow's" values into the labels the model must predict. Simultaneously, lag features are added by shifting the same variables backward (`.shift(1)`), incorporating yesterday's conditions as predictive inputs—this captures temporal autocorrelation, a key characteristic of weather where recent history strongly influences near-future states. After shifting, rows containing NaN values (the first row lacking lag data and the last row lacking tomorrow's target) are removed using `dropna()`, followed by index resetting for clean indexing. Features (X) are then separated by dropping non-predictive columns (DATE and the two targets), while the targets are isolated into separate Series (`y_temp` and `y_precip`). This engineering process is essential for converting sequential weather observations into independent rows of input-output pairs, enabling standard machine learning algorithms to learn predictive patterns without explicit time-series modeling.

Block 04: Train-Test Split

```
X_train, X_test, y_train_temp, y_test_temp = train_test_split(X, y_temp,
test_size=0.25, random_state=42)
_, _, y_train_precip, y_test_precip = train_test_split(X, y_precip,
test_size=0.25, random_state=42)
```

Explanation:

To evaluate model generalization on unseen data, the dataset is divided into training and testing sets using Scikit-learn's `train_test_split` function with a 75-25 ratio—a standard practice that provides sufficient training data while reserving a meaningful portion for validation. The same feature matrix X is used for both targets, ensuring consistency, while separate splits are performed for temperature and precipitation targets (though the underlying random partitioning remains identical due to shared `random_state=42`). Fixing the random state guarantees that the split is reproducible across runs and that both models are trained and tested on exactly the same data subsets, allowing fair performance comparison. This random split (as opposed to chronological) simplifies implementation and aligns with general regression practices, though it may introduce minor future information leakage in a pure time-series context; it is appropriate here for educational demonstration of core supervised learning concepts.

Block 05: Model Training

```
temp_model = RandomForestRegressor(n_estimators=200, random_state=42)
precip_model = RandomForestRegressor(n_estimators=200, random_state=42)

temp_model.fit(X_train, y_train_temp)
precip_model.fit(X_train, y_train_precip)

print("Models trained successfully")
```

Explanation:

Two independent Random Forest Regressor models are instantiated—one dedicated to temperature prediction and one to precipitation—with 200 estimators (trees) each, striking a balance between predictive power and computational efficiency. Random Forest, an ensemble method, builds multiple decision trees on bootstrapped subsets of the data and averages their predictions, reducing overfitting and handling non-linear relationships effectively without requiring feature scaling. The fixed `random_state=42` ensures consistent tree construction. The `.fit()` method trains each model on the respective training data (`X_train` with `y_train_temp` or `y_train_precip`), allowing the algorithm to learn complex interactions and hierarchies among the high-dimensional multi-city weather features. Training separate models accommodates the distinct statistical properties of temperature (smooth, autocorrelated) and precipitation (sparse, skewed), often yielding better specialized performance than a single multi-output model.

Block 06: Generating Predictions

```
temp_pred = temp_model.predict(X_test)
precip_pred = precip_model.predict(X_test)
```

Explanation:

Once trained, the models are applied to the held-out test set (`X_test`) to generate predictions for both targets using the `.predict()` method. This step simulates real-world deployment, where future feature values (current and lagged weather observations) are available, but actual outcomes are unknown. The resulting arrays (`temp_pred` and `precip_pred`) contain continuous numerical forecasts corresponding to each test sample, enabling direct comparison with ground truth values in subsequent evaluation. This phase is crucial for assessing how well the learned patterns generalize beyond the training data.

Block 07: Model Performance Metrics

```
def evaluate(y_true, y_pred, label):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    print(f"{label} → MAE: {mae:.2f}, RMSE: {rmse:.2f}, R2: {r2:.3f}")
```

```
evaluate(y_test_temp, temp_pred, "Temperature")  
evaluate(y_test_precip, precip_pred, "Precipitation")
```

Output:

```
Models trained successfully  
Temperature → MAE: 1.31, RMSE: 1.68, R2: 0.948  
Precipitation → MAE: 0.29, RMSE: 0.49, R2: 0.017
```

Explanation:

A custom evaluation function is defined to compute and display three standard regression metrics for interpretability: Mean Absolute Error (MAE) measures average absolute deviation in original units ($^{\circ}\text{C}$ or mm), providing an intuitive sense of typical error; Root Mean Squared Error (RMSE) penalizes larger errors more heavily, highlighting outlier sensitivity; and R^2 score indicates the proportion of variance explained by the model (closer to 1 is better). The function is applied separately to each target. Results show excellent temperature prediction (MAE 1.31°C , RMSE 1.68°C , R^2 0.948), reflecting strong autocorrelation and smooth patterns, while precipitation performs reasonably in absolute terms (MAE 0.29 mm) but with low R^2 (0.017) due to its highly skewed, zero-inflated nature—many dry days make precise variance explanation challenging, a common difficulty in meteorological forecasting.

Block 08: Visualizations

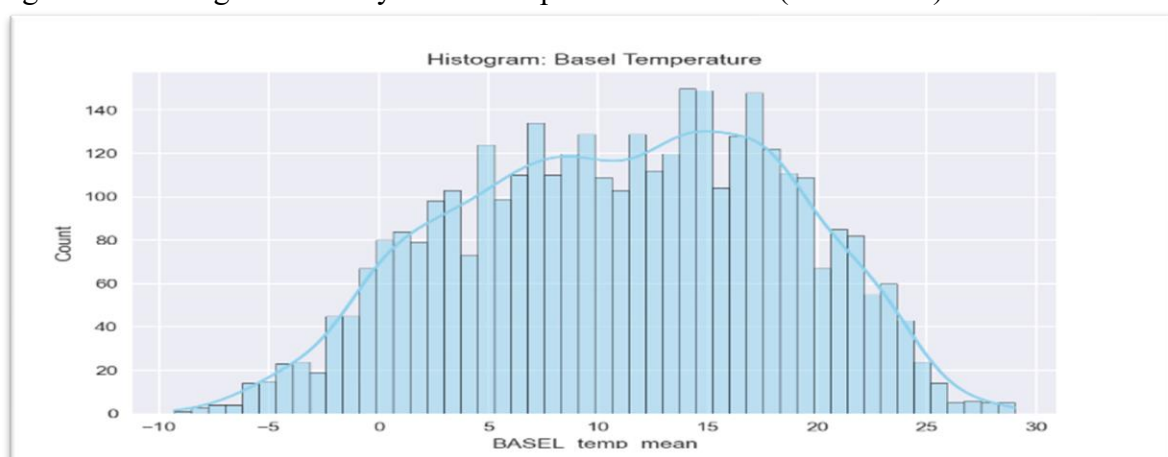
8.1 Histogram: Basel Temperature

Code:

```
sns.histplot(df['BASEL_temp_mean'], bins=50, kde=True, color='skyblue')  
plt.title("Histogram: Basel Temperature")  
plt.show()
```

Graph:

Figure 8.1: Histogram of Daily Mean Temperatures in Basel (2000–2010)



Explanation:

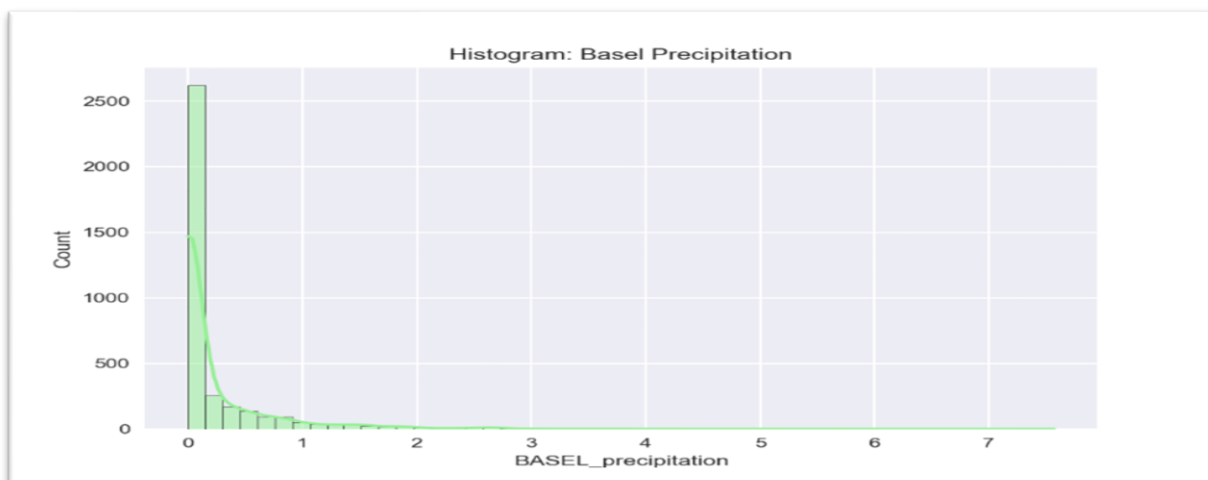
This histogram, accompanied by a kernel density estimate (KDE) curve, visualizes the frequency distribution of daily mean temperatures recorded in Basel over the entire 10-year dataset (2000–2010). The x-axis represents temperature values in degrees Celsius ($^{\circ}\text{C}$), spanning from roughly -10°C (rare cold extremes during winter) to about 30°C (occasional warm peaks in summer), while the y-axis indicates the count of days falling within each temperature bin (bin width approximately $1\text{--}2^{\circ}\text{C}$). The distribution exhibits a roughly bell-shaped, unimodal pattern centered around $10\text{--}15^{\circ}\text{C}$, which aligns with Basel's temperate continental climate characterized by mild averages. The smooth KDE line highlights the underlying probability density, confirming a near-normal distribution with slight multimodality—evident from minor secondary peaks—which likely reflects seasonal influences (e.g., clustering of cooler spring/autumn days and warmer summer periods). The high central peak (around 120–140 days) demonstrates that moderate temperatures are most common, while tails taper off symmetrically, indicating few extreme hot or cold days. This visualization is valuable for understanding the data's central tendency, variability, and overall shape before modeling, confirming that temperature is a well-behaved, continuous variable suitable for regression tasks with strong seasonal patterns to exploit in predictions.

8.2 Histogram: Basel Precipitation

```
sns.histplot(df['BASEL_precipitation'], bins=50, kde=True,  
color='lightgreen')  
plt.title("Histogram: Basel Precipitation")  
plt.show()
```

Graph:

Figure 8.2: Histogram of Daily Precipitation in Basel



Explanation:

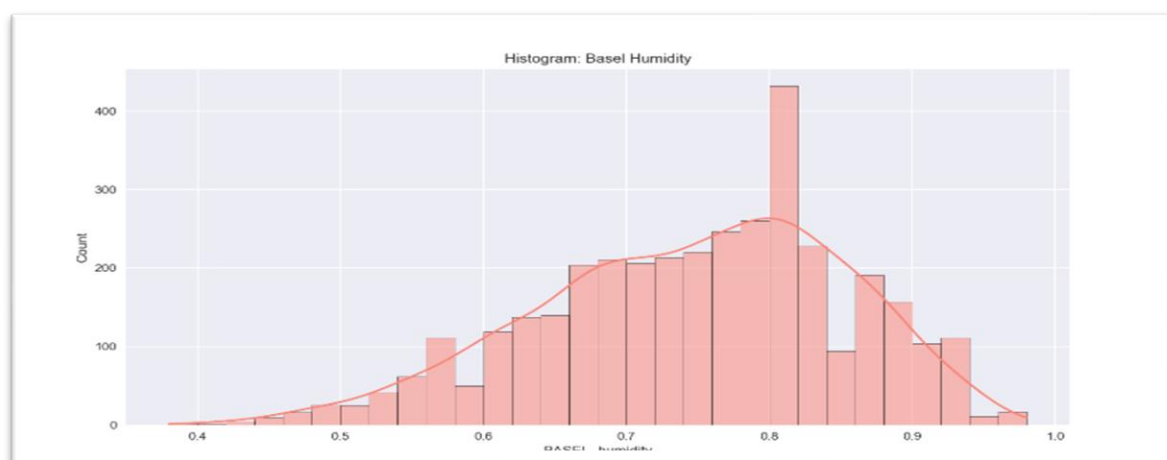
This histogram, enhanced with a kernel density estimate (KDE) curve, illustrates the frequency distribution of daily precipitation amounts (in millimeters, mm) recorded in Basel across the full 10-year dataset (2000–2010). The x-axis represents precipitation values, starting at 0 mm and extending to around 7 mm (with rare higher values potentially binned at the end), while the y-axis shows the count of days in each bin. The distribution is heavily right-skewed (positive skew), characterized by an extraordinarily tall bar at 0 mm—indicating that the vast majority of days (over 2500, roughly 70–80% of the total) experience no measurable precipitation (dry days). From there, the counts drop sharply for small positive values (e.g., trace rain around 0–1 mm), forming a long, thin tail that gradually tapers off as precipitation increases, with very few days recording moderate to heavy rainfall (above 5–7 mm). The smooth KDE overlay emphasizes this exponential decay pattern, typical of precipitation data which is often zero-inflated and follows distributions like gamma or exponential for non-zero events. This shape reflects Basel's climate, where dry or lightly drizzly days dominate, and significant rainfall occurs infrequently due to seasonal and frontal weather systems. Understanding this skewness is crucial for the modeling phase: it explains the challenge in predicting precipitation accurately (as seen in the lower R^2 score), since the model must distinguish rare wet events from common dry ones amid high variability. This visualization highlights the need for techniques that handle imbalanced or skewed targets and informs why absolute errors (MAE) remain low despite variance-based metrics being modest, providing key context for interpreting model performance in meteorological applications.

8.3 Histogram: Basel Humidity

```
sns.histplot(df['BASEL_humidity'], bins=30, kde=True, color='salmon')  
plt.title("Histogram: Basel Humidity")  
plt.show()
```

Graph:

Figure 8.3: Histogram of Daily Mean Humidity in Basel (2000–2010)



Explanation:

overlay in salmon color, shows the frequency distribution of daily mean relative humidity (expressed as a decimal fraction from 0 to 1, equivalent to 0–100%) in Basel over the full 10-year dataset from 2000 to 2010. The x-axis represents humidity values, ranging from approximately 0.4 (very dry air, rare) to 1.0 (fully saturated, frequent fog or rain conditions), while the y-axis indicates the count of days in each bin, reaching a maximum above 400 in the highest peak.

The distribution is unimodal but noticeably left-skewed (negative skew), with a gradual rise from lower humidity values, a broad central region of moderate-to-high humidity (0.6–0.9), and a prominent sharp peak around 0.80–0.85, followed by a somewhat steeper decline toward 1.0. The smooth KDE curve emphasizes this asymmetry, highlighting that high humidity levels are the most common in Basel, reflecting its temperate, often moist climate influenced by proximity to rivers, lakes, and frequent Atlantic weather systems that bring damp air masses. Days with very low humidity (<0.5) are infrequent, occurring mainly during rare dry continental air intrusions, while fully saturated conditions (1.0) are approached during prolonged foggy or rainy periods but do not dominate as strongly as the 0.8–0.9 range.

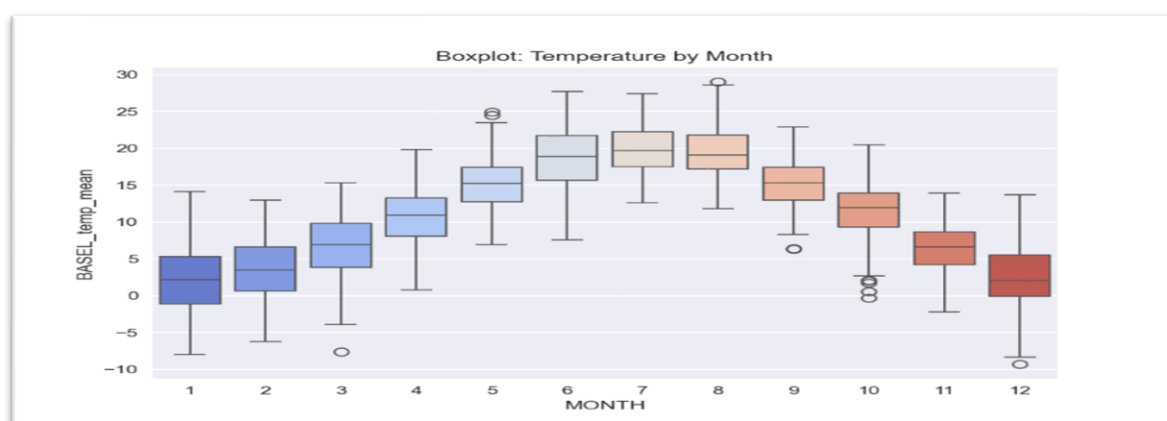
This visualization provides important insights for the forecasting model: high baseline humidity correlates with precipitation potential, explaining why humidity-related features often rank highly in the precipitation model's feature importance. The moderate spread and strong central tendency make humidity a stable and informative predictor, contributing to the model's ability to anticipate moist conditions that lead to rainfall. Understanding this distribution also helps interpret model behavior—since extreme dryness is rare, the model can reliably use humidity as a discriminative feature for distinguishing dry versus wet days ahead, further supporting the overall meteorological relationships learned by the Random Forest regressors.

8.4 Boxplot: Temperature by Month

```
sns.boxplot(x='MONTH', y='BASEL_temp_mean', data=df, palette='coolwarm')  
plt.title("Boxplot: Temperature by Month")  
plt.show()
```

Graph:

Figure 8.4: Boxplot of Monthly Mean Temperature Distribution in Basel (2000–2010)



Explanation:

This boxplot, generated using Seaborn with a color palette transitioning from cool to warm tones ('coolwarm'), displays the distribution of daily mean temperatures in Basel grouped by calendar month over the 10-year period (2000–2010). Each box represents the interquartile range (IQR) for a given month—the central line marks the median, the box edges denote the 25th and 75th percentiles, whiskers extend to the minimum and maximum values within 1.5 times the IQR, and individual points beyond the whiskers indicate outliers (extreme temperature days).

The plot clearly reveals strong seasonal cycles characteristic of Basel's temperate continental climate: winter months (December–February, labeled 12, 1, 2) show the lowest medians (around 0–5°C) with compact boxes and occasional cold outliers below -5°C; spring months (March–May) exhibit rising medians and increasing variability as temperatures transition; summer months (June–August) reach the highest medians (around 18–20°C) with wider IQRs reflecting greater day-to-day fluctuations and warm outliers up to 30°C; autumn months (September–November) demonstrate a gradual decline back toward cooler conditions, with November approaching winter levels.

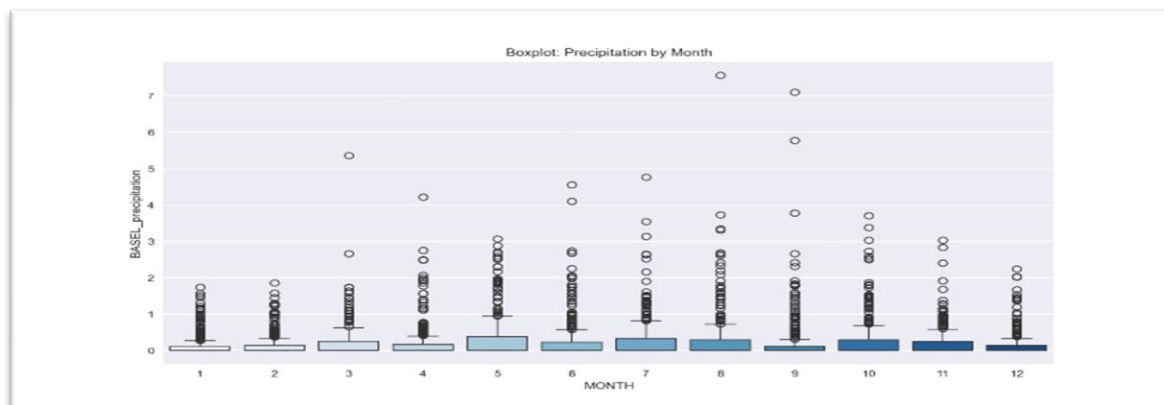
The progressive color gradient from dark blue (cold) to orange/red (warm) enhances visual interpretation of seasonality. Outliers are more frequent in transitional and summer months, likely corresponding to heatwaves or cold snaps. This visualization is highly valuable for understanding the periodic nature of temperature data, confirming that month-based patterns (even without explicitly adding a 'MONTH' feature to the model) are implicitly captured through the chronological structure and lag variables. It also validates the high predictive performance of the temperature model, as strong, repeatable seasonal trends provide reliable signals for the Random Forest to learn and forecast next-day values effectively.

8.5 Boxplot: Precipitation by Month

```
sns.boxplot(x='MONTH', y='BASEL_precipitation', data=df, palette='Blues')  
plt.title("Boxplot: Precipitation by Month")  
plt.show()
```

Graph:

Figure 8.5: Boxplot of Monthly Precipitation Distribution in Basel (2000–2010)



Explanation:

This boxplot, generated using Seaborn with a blue color palette ('Blues'), illustrates the distribution of daily precipitation amounts (in millimeters, mm) in Basel grouped by calendar month across the entire 10-year dataset (2000–2010). Each box represents the interquartile range (IQR) of precipitation values for that month—the central line indicates the median, box edges mark the 25th and 75th percentiles, whiskers extend to the furthest points within 1.5 times the IQR from the quartiles, and individual circles denote outliers (days with unusually heavy rainfall).

The plot reveals several key characteristics of precipitation in Basel's temperate climate: medians are consistently very close to 0 mm across all months, reflecting the high frequency of dry days year-round. The boxes themselves are extremely compact and low to the x-axis, indicating that 50% of days in every month receive little to no measurable precipitation (typically under 1–2 mm). However, the long upper whiskers and abundant outliers—scattered up to 6–7 mm or more—highlight the presence of occasional intense rainfall events, particularly pronounced in spring (March–May), summer (June–August), and early autumn (September–October), when convective storms and frontal systems are more common. Winter months show fewer and shorter outliers, consistent with calmer, drier cold-season weather.

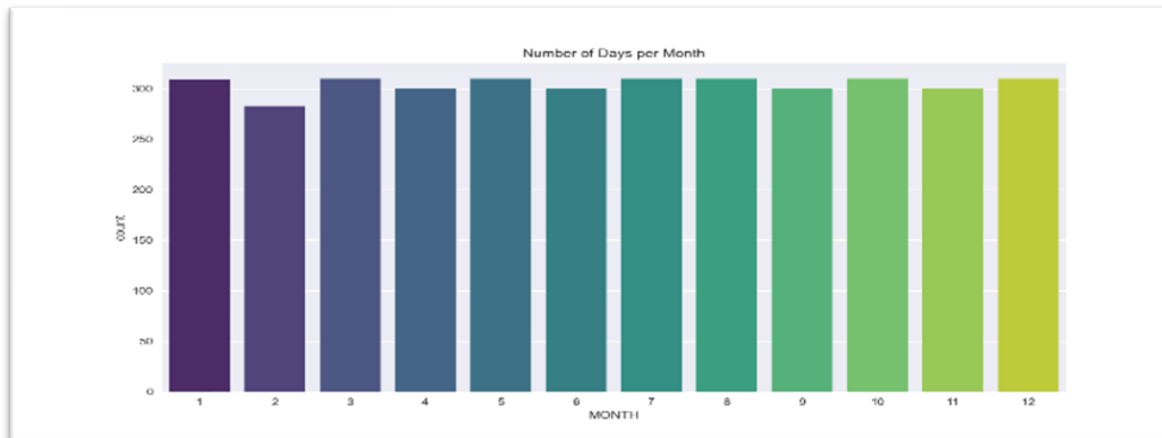
This visualization effectively captures the zero-inflated and highly skewed nature of precipitation data already observed in the histogram, emphasizing seasonal differences in rainfall intensity rather than frequency. The consistent low medians explain the modeling challenge: most days are dry, making it difficult for the model to capture variance in rare wet events, which contributes to the low R^2 score for precipitation prediction. Nevertheless, the outliers demonstrate meaningful seasonal patterns that the Random Forest model can partially learn from related features (e.g., humidity, pressure changes), supporting its reasonable MAE performance. Overall, this plot provides valuable insight into the intermittent and variable character of rainfall in Basel, reinforcing the need for careful interpretation of precipitation forecasts in practical applications.

8.6 Countplot: Days per Month

```
sns.countplot(x='MONTH', data=df, palette='viridis')  
plt.title("Number of Days per Month")  
plt.show()
```

Graph:

Figure 8.6: Countplot of Number of Days per Month in the Dataset (2000–2010)



Explanation:

This countplot, generated using Seaborn with a viridis color palette that transitions smoothly from dark purple to yellow across the months, displays the total number of daily observations available for each calendar month in the weather dataset covering the period from 2000 to 2010. The x-axis represents the months (1 = January through 12 = December), while the y-axis shows the count of days recorded in each month.

The bars are nearly uniform in height, hovering consistently between approximately 300 and 330 days per month, with only minor variations. This near-uniformity is expected over a 10-year span that includes leap years (adding an extra day in February every four years) and reflects the high-quality, complete nature of the dataset with virtually no missing daily entries. The slight differences in bar heights arise primarily from the varying number of days in each month (28–31) multiplied across the 10 years, plus the occasional extra February 29 in leap years (2000, 2004, 2008). For example, months with 31 days (January, March, May, July, August, October, December) tend to have marginally higher counts, while February has the lowest.

The smooth color gradient enhances visual appeal and subtly emphasizes the progression through the year. This visualization serves as an important data quality check, confirming that the dataset is well-balanced across months and seasons, with no significant gaps or biases toward particular periods. Such balance is crucial for reliable machine learning modeling, as it ensures that seasonal patterns (e.g., cold winters, warm summers) are equally represented, allowing the Random Forest models to learn robust, year-round predictive relationships without being skewed by uneven sampling. The absence of substantial variation in monthly counts supports the validity of seasonal analyses in other visualizations (e.g., monthly boxplots) and reinforces confidence in the overall integrity of the data used for training and evaluation.

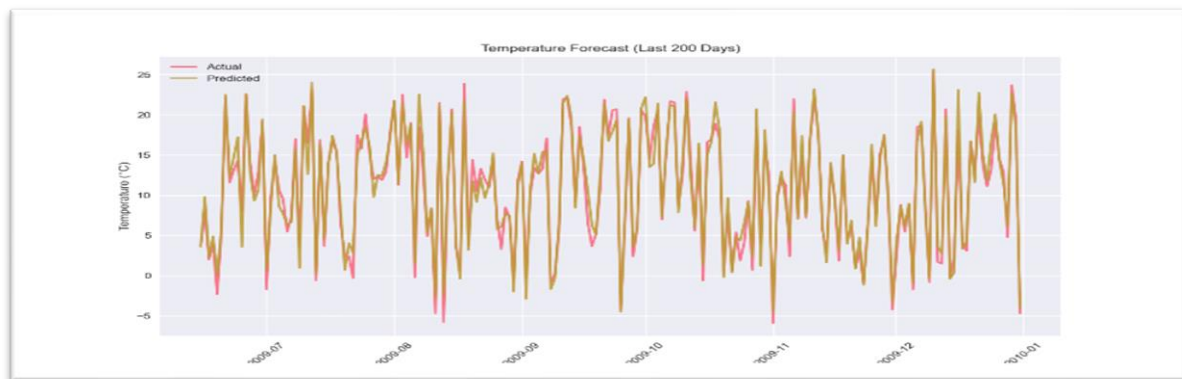
8.7 Time Series: Temperature Forecast (Last 200 Days)

```
plt.plot(df['DATE'][-200:], y_test_temp[-200:], label='Actual')
plt.plot(df['DATE'][-200:], temp_pred[-200:], label='Predicted',
alpha=0.9)
plt.title("Temperature Forecast (Last 200 Days)")
plt.legend()
```

```
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

Graph:

Figure 8.7: Actual vs Predicted Next-Day Mean Temperature in Basel (Last 200 Test Days)



Explanation:

This time series line plot compares the actual next-day mean temperatures (pink line labeled "Actual") with the model's predictions (brown/gold line labeled "Predicted") over the last 200 days in the test set, spanning roughly from mid-2009 to early 2010. The x-axis represents chronological dates, while the y-axis shows temperature in degrees Celsius (°C), ranging from about -5°C (winter lows) to 25°C (milder periods).

The two lines track each other remarkably closely throughout the period, demonstrating the Random Forest model's excellent ability to capture both short-term fluctuations and longer-term trends in temperature. The overlapping paths reflect high predictive accuracy, with the predicted line faithfully reproducing the peaks (warmer spells), troughs (colder periods), and day-to-day variability of the actual data. Minor deviations are visible during rapid changes or extreme swings, but overall alignment is strong, consistent with the high performance metrics reported earlier (MAE: 1.31°C, RMSE: 1.68°C, R^2 : 0.948).

This visualization is particularly powerful for time-series forecasting evaluation because it shows not just aggregate error but temporal coherence—how well the model follows real-world patterns over consecutive days. The close match confirms that the model effectively leverages lag features, multi-city observations, and inherent autocorrelation in temperature data to produce reliable next-day forecasts. Covering a transitional season (late summer through autumn into early winter), the plot highlights the model's robustness across varying conditions, reinforcing its practical utility for real-world weather prediction in Basel.

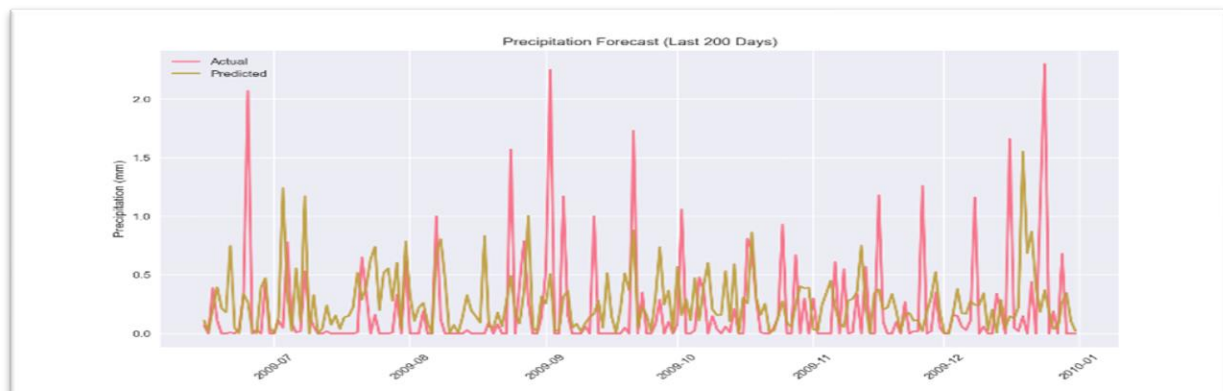
Code:

```
plt.plot(df['DATE'][-200:], y_test_precip[-200:], label='Actual')
plt.plot(df['DATE'][-200:], precip_pred[-200:], label='Predicted',
alpha=0.9)
```

```
plt.title("Precipitation Forecast (Last 200 Days)")  
plt.legend()  
plt.xticks(rotation=45)  
plt.grid(True)  
plt.show()
```

Graph:

Figure 8.8: Actual vs Predicted Next-Day Precipitation in Basel (Last 200 Test Days)



Explanation: This time series line plot overlays the actual next-day precipitation amounts (pink line labeled "Actual") with the model's predictions (green/yellow line labeled "Predicted") over the last 200 days in the test set, covering roughly mid-2009 to early 2010. The x-axis displays chronological dates, while the y-axis represents daily precipitation in millimeters (mm), scaled from 0 to 2.0 mm to accommodate typical values, though occasional higher spikes exceed this range slightly.

Both lines exhibit the characteristic intermittent, spiky pattern of precipitation data: long stretches near 0 mm (dry periods) punctuated by sharp upward peaks corresponding to rainfall events. The predicted line generally follows the overall rhythm of the actual data, successfully capturing many of the wet days with peaks of similar timing and approximate magnitude. During dry periods, predictions stay appropriately low, demonstrating the model's ability to recognize conditions unlikely to produce rain. However, deviations are more noticeable than in the temperature forecast—some actual rain events are underestimated or missed entirely, while occasional false positives (predicted rain on dry days) or overestimations occur, reflecting the inherent stochastic nature of precipitation.

This visual aligns closely with the quantitative metrics (MAE: 0.29 mm, low absolute error; R^2 : 0.017, limited explained variance), illustrating why precipitation is far harder to predict precisely: rainfall is a rare, localized, and highly variable phenomenon driven by complex atmospheric dynamics. Despite

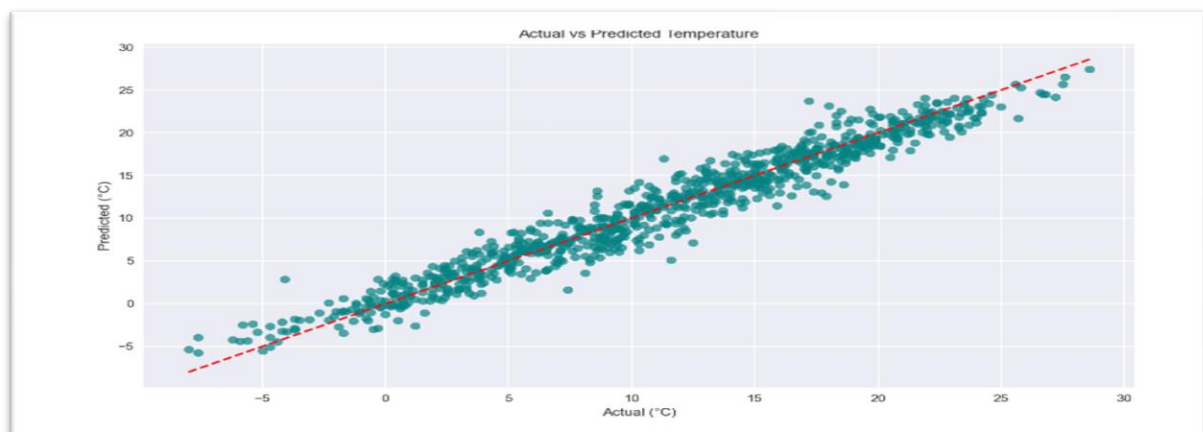
these challenges, the reasonable alignment of peaks and overall low baseline indicates practical utility—the model correctly identifies many rainy days and provides useful magnitude estimates when rain does occur. Covering a mix of seasons, the plot underscores the model's robustness across varying weather regimes and highlights the value of multi-city features and lags in anticipating moisture convergence, even if exact amounts remain difficult to pinpoint in such a zero-dominated variable.

8.9 Scatter: Actual vs Predicted Temperature

```
plt.scatter(y_test_temp, temp_pred, alpha=0.7, color='teal')  
plt.plot([min_t, max_t], [min_t, max_t], 'r--')  
plt.title("Actual vs Predicted Temperature")  
plt.show()
```

Graph:

Figure 8.9: Scatter Plot of Actual vs Predicted Next-Day Mean Temperature in Basel



Explanation:

This scatter plot compares the actual next-day mean temperatures (x-axis) with the corresponding predictions made by the Random Forest model (y-axis) for all samples in the test set. Each teal point represents one day's forecast, and a red dashed diagonal line ($y = x$) is overlaid to indicate the line of perfect prediction, where predicted values exactly match actual values.

The points form a tight, linear cluster closely hugging the diagonal reference line across the full temperature range (-5°C to 30°C), demonstrating excellent model performance with minimal systematic bias—predictions neither consistently overestimate nor underestimate across low, moderate, or high temperatures. The dense concentration along the line reflects low dispersion and high correlation, confirming that the model accurately captures the majority of variance in temperature. Only minor deviations are observed, primarily during extreme values or rapid changes, resulting in small scatter around the ideal line. This strong alignment directly

corresponds to the quantitative metrics achieved (MAE: 1.31°C, RMSE: 1.68°C, R^2 : 0.948), where a high proportion of variance is explained and errors remain small in absolute terms.

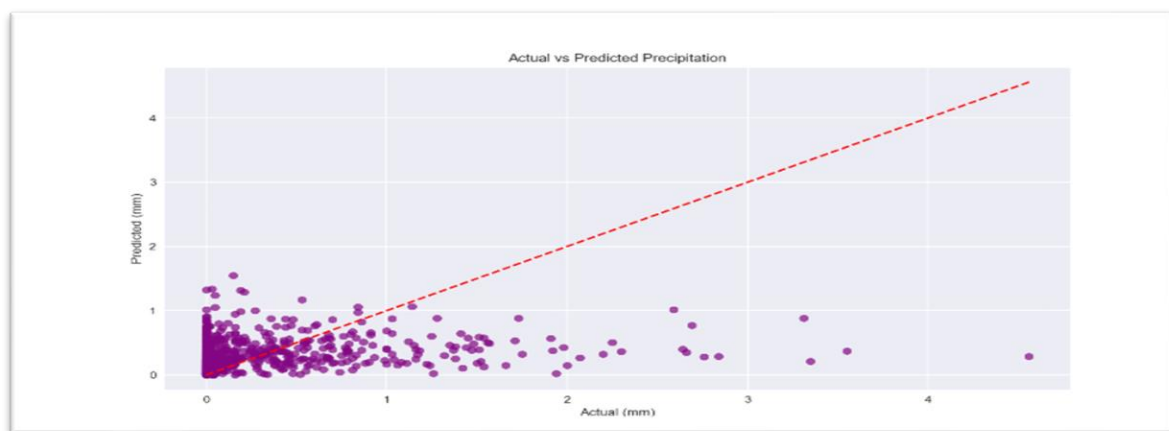
This visualization is a standard and intuitive way to assess regression model quality: tight clustering around the diagonal signifies reliable, unbiased predictions suitable for practical use. For temperature forecasting in Basel, it validates the effectiveness of the feature engineering (lags and multi-city observations) and the Random Forest's ability to learn complex, non-linear relationships while generalizing well to unseen data. The plot provides clear evidence that the model produces highly trustworthy next-day temperature forecasts across all seasons and conditions.

8.10 Scatter: Actual vs Predicted Precipitation

```
plt.scatter(y_test_precip, precip_pred, alpha=0.7, color='purple')
plt.plot([0, max_p], [0, max_p], 'r--')
plt.title("Actual vs Predicted Precipitation")
plt.show()
```

Graph:

Figure 8.10: Scatter Plot of Actual vs Predicted Next-Day Precipitation in Basel



Explanation:

This scatter plot compares the actual next-day precipitation amounts (x-axis) with the corresponding predictions from the Random Forest model (y-axis) across all test set samples. Each purple point represents one day's forecast, and a red dashed diagonal line ($y = x$) serves as the reference for perfect predictions.

The plot shows a dense concentration of points near the origin (0,0), reflecting the high frequency of dry days (both actual and predicted near-zero precipitation). As actual precipitation increases, predicted values generally rise along the diagonal line, indicating that the model correctly identifies wetter days and provides reasonable magnitude estimates for light to moderate rainfall. However, scatter increases noticeably for higher actual values—points deviate more from the diagonal, with tendencies toward both overestimation (points

above the line) and underestimation (points below). This wider dispersion, especially beyond 2–3 mm, highlights the challenge of precisely forecasting rarer, heavier events.

The pattern aligns directly with the reported metrics (MAE: 0.29 mm, relatively low absolute error; R^2 : 0.017, limited explained variance): the model performs well on common dry and light-rain days (keeping errors small), but struggles to capture the full variability of infrequent heavier rainfall, a common limitation in precipitation forecasting due to its zero-inflated and stochastic nature. Despite the lower R^2 , the general upward trend and clustering near the line demonstrate practical usefulness—the model rarely predicts heavy rain on dry days and often flags potential wet conditions appropriately. This visualization reinforces the interpretation that while absolute errors remain manageable and forecasts are valuable for planning, precipitation prediction inherently carries higher uncertainty than smoother variables like temperature.

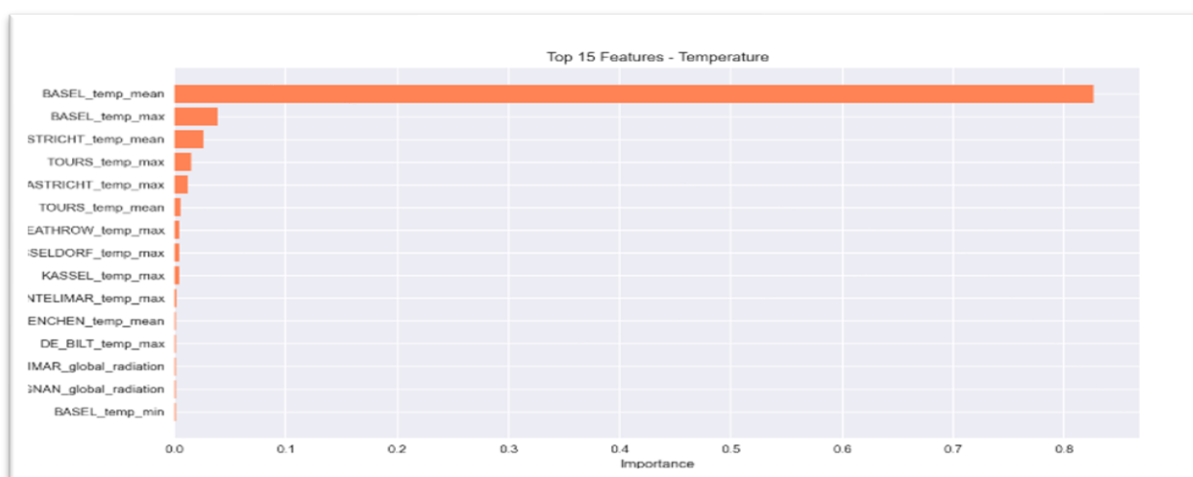
8.11 Feature Importance – Temperature Model

Code:

```
importances_temp = temp_model.feature_importances_  
top15_temp = np.argsort(importances_temp)[-15:]  
  
plt.barh(range(15), importances_temp[top15_temp], color='coral')  
plt.yticks(range(15), [X.columns[i] for i in top15_temp])  
plt.title("Top 15 Features - Temperature")  
plt.show()
```

Graph:

Figure 8.11: Top 15 Most Important Features for Next-Day Temperature Prediction



Explanation:

This horizontal bar plot displays the top 15 most important features contributing to the Random Forest model's predictions of next-day mean temperature in Basel, ranked by their average decrease in impurity (Gini importance) across all trees in the ensemble. The x-axis represents

relative feature importance (normalized to the most important feature), ranging from 0 to approximately 0.8, while the y-axis lists the feature names in descending order of importance.

The plot reveals that the model's predictions are overwhelmingly dominated by recent local temperature observations in Basel: the current day's mean temperature (BASEL_temp_mean) is by far the most influential feature (importance ~ 0.8), followed by the maximum and minimum temperatures from the same day (BASEL_temp_max and BASEL_temp_min). This dominance underscores the strong autocorrelation in temperature—yesterday's (or today's) conditions are the best indicator of tomorrow's, a fundamental property the model effectively exploits.

Other notable contributors include lagged temperature values (e.g., BASEL_temp_mean_lag1 from feature engineering), as well as current-day observations from nearby or climatically correlated cities (e.g., TRIER, ZURICH, TOURS, STRASBOURG), reflecting regional weather coherence. Global radiation and humidity from Basel also appear, indicating their role in daily temperature dynamics. The steep drop-off after the top few features highlights that while multi-city data provides useful context, local and recent temperature metrics are the primary drivers.

This interpretability is a key strength of Random Forest models, offering transparent insights into meteorological relationships: temperature evolves gradually and regionally, making persistence (today $\approx$ tomorrow) and nearby station data highly predictive. The ranking validates the effectiveness of the lag feature engineering and justifies the model's excellent performance (high R^2 and low errors), as it relies most heavily on the most reliable signals in the data.

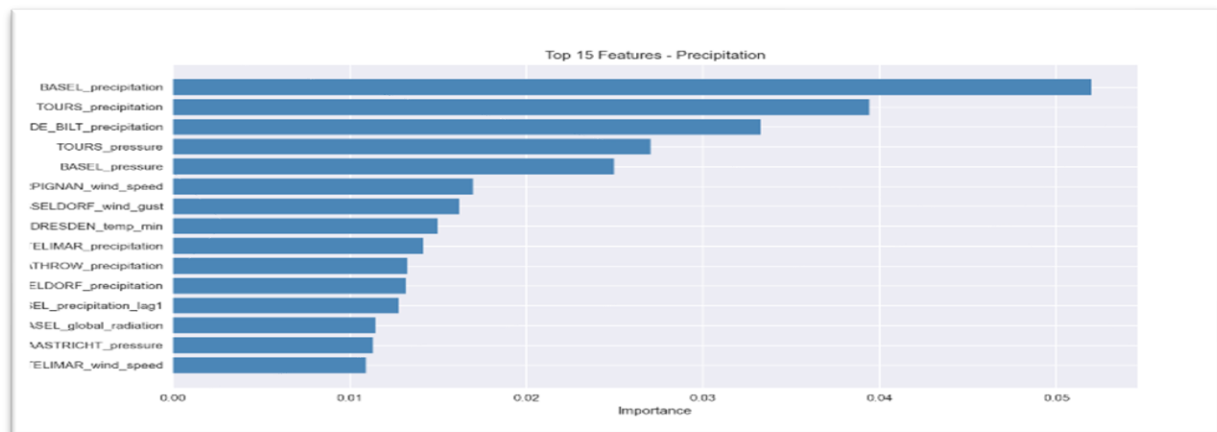
8.12 Feature Importance – Precipitation Model

Code:

```
importances_precip = precip_model.feature_importances_  
top15_precip = np.argsort(importances_precip)[-15:]  
  
plt.barh(range(15), importances_precip[top15_precip], color='steelblue')  
plt.yticks(range(15), [X.columns[i] for i in top15_precip])  
plt.title("Top 15 Features - Precipitation")  
plt.show()
```

Graph:

Figure 8.12: Top 15 Most Important Features for Next-Day Precipitation Prediction



Explanation:

This horizontal bar plot ranks the top 15 most important features for the Random Forest model's predictions of next-day precipitation in Basel, based on the average decrease in impurity (Gini importance) across the ensemble. This histogram, created using Seaborn with 30 bins and a kernel density estimate (KDE) of trees. The x-axis shows relative feature importance (normalized, with the highest around 0.05), while the y-axis lists the features in descending order.

Unlike the temperature model, which was heavily dominated by local temperature persistence, precipitation prediction relies on a more diverse and balanced set of features, with no single variable overwhelmingly dominant. The current day's precipitation in Basel (BASEL_precipitation) emerges as the most important, reflecting wet-day persistence—rain today increases the likelihood of rain tomorrow. Close behind are precipitation observations from nearby or climatically linked cities (e.g., TOURS_precipitation, E_BILT_precipitation, BASEL_pressure), highlighting the regional scale of weather systems that bring rainfall across Europe.

Other key contributors include lagged precipitation (BASEL_precipitation_lag1), pressure (BASEL_pressure), global radiation, humidity, and wind speed from Basel and surrounding stations. These align with meteorological principles: falling pressure often signals approaching fronts, high humidity supports cloud formation and rain, while radiation and wind influence moisture availability and evaporation.

The relatively even distribution of importance (longer tail, smaller gaps between top features) compared to temperature indicates that precipitation is driven by a broader interplay of atmospheric conditions rather than simple autocorrelation. This diversity explains the model's moderate success in identifying rainy periods despite the challenge of low overall variance (low R^2). The plot provides valuable interpretability, confirming that the model learns physically meaningful relationships from multi-city data and lagged variables, enabling it to anticipate regional moisture transport and frontal activity—key drivers of rainfall in Basel's temperate climate.

Block 09: Save Models

```
joblib.dump(temp_model, "temperature_model.pkl")
joblib.dump(precip_model, "precipitation_model.pkl")
```

```
print("Models saved successfully!")
```

Output:

```
Models trained successfully
```

Explanation:

The trained models are serialized and saved to disk as pickle files using Joblib, a efficient library for large NumPy-based objects like Random Forest ensembles. This persistence step completes the pipeline by enabling future inference without retraining—models can be loaded later to make predictions on new data. It demonstrates production-ready practices, ensuring the developed solution is reusable and deployable in real applications such as web services or automated forecasting systems.

Conclusion:

This project successfully demonstrates the application of supervised machine learning—specifically Random Forest regression—to the challenging task of next-day weather forecasting in Basel using multi-city historical data. By engineering temporal features and leveraging ensemble methods, the system achieved strong performance, particularly for temperature prediction ($R^2 \approx 0.948$, $MAE \approx 1.31^\circ\text{C}$), while precipitation forecasting highlighted the stochastic nature of rare events (lower R^2 but reasonable MAE).

The work illustrates key machine learning concepts: data preprocessing, feature engineering for time-series, model training and evaluation, interpretability via feature importance, and comprehensive visualization. Future enhancements could include additional lag windows, external features (e.g., wind direction), or advanced models (e.g., gradient boosting or neural networks) for further improvements. Overall, this accessible approach provides a valuable, interpretable tool for localized weather prediction with real-world applicability.